MODUL PRAKTIKUM PEMROGRAMAN WEB

MODUL 6: ASYNCHRONOUS JAVASCRIPT & API FETCHING

BAB 1: PENGENALAN ASYNCHRONOUS JAVASCRIPT & FETCH API

1.1 Konsep Sinkronus vs Asinkronus

Secara bawaan, JavaScript adalah bahasa pemrograman yang bersifat single-threaded, artinya hanya dapat mengerjakan satu instruksi dalam satu waktu dari atas ke bawah. Berdasarkan cara pengeksekusiannya, terdapat dua model pemrosesan:

- **Sinkronus (Synchronous / Blocking):** Instruksi dijalankan berurutan. Instruksi berikutnya baru dapat berjalan setelah instruksi saat ini selesai. Jika ada tugas berat yang butuh waktu lama (seperti mendownload data dari internet), seluruh antarmuka web akan 'membeku' (freeze) dan tidak merespons klik pengguna.
- **Asinkronus (Asynchronous / Non-Blocking):** Tugas-tugas berat dijalankan di latar belakang (background). Program utama dapat terus berjalan melayani interaksi pengguna tanpa harus menunggu tugas latar belakang tersebut selesai.

1.2 Memahami Promise & Tiga Statusnya

Untuk menangani proses yang butuh waktu (asinkronus), JavaScript menyediakan objek bernama Promise. Promise diibaratkan seperti 'janji' yang hasilnya baru akan diketahui di masa mendatang. Sebuah Promise memiliki 3 status:

- **1. Pending:** Status awal ketika operasi asinkronus sedang berjalan dan belum selesai.
- **2. Fulfilled (Resolved):** Operasi berhasil diselesaikan dan mengembalikan data hasil proses.
- **3. Rejected:** Operasi mengalami kegagalan (misalnya koneksi internet terputus atau server mati).

1.3 Sintaks Modern: Async / Await & Penanganan Error (try...catch)

Pada standar JavaScript modern (ES6+), penulisan Promise dipermudah dengan kata kunci async dan await sehingga terlihat rapi menyerupai kode biasa:

- **async:** Diletakkan di depan deklarasi fungsi untuk menandakan bahwa fungsi tersebut berjalan secara asinkronus.
- **await:** Hanya dapat digunakan di dalam fungsi async. Bertugas menunggu sampai sebuah Promise selesai sebelum melanjutkan ke baris berikutnya.
- **try...catch:** Blok pengaman wajib untuk menangkap error. Kode di dalam blok 'try' akan dijalankan, dan jika terjadi kegagalan (seperti URL salah atau internet mati), alur program langsung dialihkan ke blok 'catch' tanpa membuat aplikasi rusak (crash).

```
// Struktur Dasar Fungsi Async/Await dengan try...catch:
const ambilData = async () => {
    try {
        console.log("Mulai mengambil data...");
        const response = await fetch("https://jsonplaceholder.typicode.com/users");
        const data = await response.json();
        console.log("Data berhasil didapatkan:", data);
    } catch (error) {
        console.error("Terjadi kesalahan koneksi:", error.message);
    }
};
```

1.4 Web API, Format JSON, dan Metode Fetch()

Web API (Application Programming Interface) adalah jembatan komunikasi antara aplikasi web kita dengan server jarak jauh. Data yang dipertukarkan biasanya berupa format JSON (JavaScript Object Notation), yaitu teks ringan yang mudah diubah menjadi objek JavaScript.

Peramban modern menyediakan fungsi bawaan fetch(url) untuk mengambil data dari server melalui 2 langkah utama:

- **Langkah 1 (Kirim Permintaan):** const response = await fetch(url); — Mengirim permintaan HTTP ke server.
- **Langkah 2 (Ekstraksi JSON):** const data = await response.json(); — Mengubah data teks JSON menjadi array/objek yang siap ditampilkan di web.
- **Pemeriksaan response.ok:** Properti bernilai true jika server merespons dengan kode sukses (HTTP status 200–299). Jika false (misal 404 Not Found), kita dapat memicu error secara manual.

BAB 2: PANDUAN PRAKTIK TERSTRUKTUR (USER DIRECTORY APP)

Pada praktikum ini, kita akan membuat aplikasi 'Daftar Pengguna' (User Directory App) yang mengambil data nyata dari internet menggunakan API publik gratis dan stabil: https://jsonplaceholder.typicode.com/users.

Langkah 2.1: Menyiapkan Struktur Halaman (index.html)

Buat folder baru bernama modul-6 di dalam proyek Anda. Di dalam folder modul-6, buat berkas index.html berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Modul 6 - User Directory App</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
```

```
        <h2>Daftar Pengguna (API Fetching)</h2>
        <p class="subtitle">Praktikum Asynchronous JavaScript & Fetch API</p>

        <!-- Tombol Aksi -->
        <div class="actions-bar">
            <button id="btnFetch" class="btn-primary">Muat Data Pengguna</button>
        </div>

        <!-- Indikator Status (Loading & Pesan Error) -->
        <div id="statusMessage" class="status-msg">Klik tombol di atas untuk memuat
data pengguna.</div>

        <!-- Kontainer Kartu Pengguna -->
        <div id="userList" class="user-grid"></div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

Langkah 2.2: Menata Tampilan Antarmuka (style.css)

Buat berkas style.css di dalam folder modul-6, lalu ketikkan kode CSS berikut:

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f8fafc; color: #1e293b; padding: 40px 20px;
    display: flex; justify-content: center;
}
.container {
    background: #ffffff; width: 100%; max-width: 650px; padding: 24px;
    border-radius: 12px; border: 1px solid #e2e8f0; box-shadow: 0 4px 12px
rgba(0,0,0,0.06);
}
h2 { color: #4338ca; text-align: center; margin-bottom: 4px; font-size: 22px; }
.subtitle { color: #64748b; text-align: center; font-size: 13px; margin-bottom: 20px;
}
.actions-bar { display: flex; justify-content: center; margin-bottom: 16px; }
button.btn-primary {
    background-color: #4338ca; color: white; border: none; padding: 10px 20px;
    border-radius: 6px; font-weight: bold; cursor: pointer; font-size: 14px;
    transition: background-color 0.2s;
}
button.btn-primary:hover { background-color: #3730a3; }
button:disabled { background-color: #94a3b8; cursor: not-allowed; }
.status-msg {
    text-align: center; font-size: 14px; color: #64748b; padding: 12px;
    border-radius: 6px; background-color: #f1f5f9; margin-bottom: 16px;
}
.status-msg.error { background-color: #fef2f2; color: #dc2626; border: 1px solid
#fecaca; }
.user-grid {
```

```
    display: grid; grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 12px;
}
.user-card {
    background: #ffffff; border: 1px solid #e2e8f0; border-radius: 8px; padding: 14px;
}
.user-name { font-size: 15px; font-weight: bold; color: #0f172a; margin-bottom: 4px; }
.user-email { font-size: 13px; color: #0284c7; margin-bottom: 4px; }
.user-company { font-size: 12px; color: #64748b; }
```

Langkah 2.3: Menulis Kode Logika Fetch API (script.js)

Buat berkas script.js di dalam folder modul-6, lalu ketikkan kode JavaScript modern berikut:

```
// File: script.js
document.addEventListener("DOMContentLoaded", () => {
    // 1. Tangkap elemen DOM
    const btnFetch = document.querySelector("#btnFetch");
    const statusMessage = document.querySelector("#statusMessage");
    const userList = document.querySelector("#userList");

    // 2. Fungsi Asinkronus untuk Mengambil Data Pengguna dari Server API
    const fetchUsers = async () => {
        // A. Tampilkan status loading saat proses dimulai
        btnFetch.disabled = true;
        statusMessage.textContent = "Sedang memuat data dari server...";
        statusMessage.className = "status-msg";
        userList.innerHTML = "";

        try {
            // B. Kirim HTTP Request ke URL API
            const response = await
fetch("https://jsonplaceholder.typicode.com/users");

            // C. Cek apakah server merespons dengan sukses
            if (!response.ok) {
                throw new Error(`Gagal mengambil data! Status HTTP:
${response.status}`);
            }

            // D. Konversi respons JSON menjadi array objek JavaScript
            const users = await response.json();

            // E. Render data pengguna ke tampilan kartu HTML
            statusMessage.textContent = `Berhasil memuat ${users.length} data
pengguna!`;
            renderUsers(users);

        } catch (error) {
            // F. Tangani jika terjadi error jaringan atau URL salah
            statusMessage.textContent = `Gagal memuat data: ${error.message}`;
            statusMessage.className = "status-msg error";
```

```
        } finally {
            // G. Aktifkan kembali tombol setelah proses selesai
            btnFetch.disabled = false;
        }
    };

    // 3. Fungsi Render Kartu Pengguna ke DOM
    const renderUsers = (users) => {
        userList.innerHTML = "";
        users.forEach(user => {
            const card = document.createElement("div");
            card.className = "user-card";
            card.innerHTML = `
                <div class="user-name">${user.name}</div>
                <div class="user-email">${user.email}</div>
                <div class="user-company">${user.company.name}</div>
            `;
            userList.appendChild(card);
        });
    };

    // 4. Tambahkan Event Listener pada tombol
    btnFetch.addEventListener("click", fetchUsers);
});
```

Langkah 2.4: Uji Coba di Browser & Simpan ke GitHub

1. Buka berkas index.html di browser. Pastikan komputer terhubung ke jaringan internet.

2. Klik tombol 'Muat Data Pengguna'. Perhatikan teks loading muncul sejenak, lalu 10 kartu pengguna berhasil ditampilkan.

3. Buka Console DevTools (F12) untuk melihat tidak ada error berwarna merah.

4. Simpan seluruh kemajuan pekerjaan ke Git dengan perintah terminal:

```
git status
git add .
git commit -m "feat: menyelesaikan praktik modul 6 asynchronous dan fetch api"
git push origin main
```